Panchip Microelectronics Co., Ltd.

PAN3029/3060 Series SDK User Guide

Current version: 0.6

Release date: 2024.04

Shanghai Panqi Microelectronics Co., Ltd.

Address: Room 302, Building D, No. 666, Shengxia Road, Zhangjiang Hi-Tech Park, Shanghai Tel: 021-50802371

Website: http://www.panchip.com

Documentation

Due to version upgrades or other reasons, the content of this document will be updated from time to time. Unless otherwise agreed, the content of this document is only for guidance, and all statements, information and suggestions in this document do not constitute any express or implied warranty.

Trademark

Panqi is a trademark of Panqi Microelectronics Co., Ltd. Other names mentioned in this document are trademarks/registered trademarks of their respective owners

Disclaimer

All or part of the products, services or features described in this document may not be within the scope of your purchase or use. Unless otherwise agreed in the contract, Panqi Microelectronics Co., Ltd. makes no express or implied statement or warranty regarding the contents of this document.

Revision History

Version	Revision date	Description
V0.1	2023.06	Initial version
V0.2	2023.08	1.Modify the operation process after using single packet mode 2.Added interface for obtaining current channel energy detection
V0.3	2023.09	1.Update transmit power table 2.Update MAPM interface
V0.4	2023.12	1.Update transmit power table 1-23 Gear
V0.5	2023.12	1.Update SDK Migration Instructions
V0.6	2024.04	Modify function interface and chip description

目录

1 Overview

This document mainly introduces the SDK interface functions of the PAN3029/3060 series chips, including the chip initialization process and the data transmitting and receiving process.

SDK related files include "pan_rf.c", "pan_rf.h", "pan_port.c", "pan_port.h", and "pan_param.h".

2 SDK Usage

This chapter describes in detail the basic process of using the SDK, including the most basic initialization, parameter configuration, and data transmission and reception.

2.1 Initialization

2.1.1 Interface

```
rf_init();
```

2.1.2 Implementation

1. Wake up the chip from deep sleep state to standby3 state
2. Load FT configuration
3. Initialize register configuration
4. AGC Configuration
5. Initialize chip TX_IO/RX_IO

2.1.3 Usage

Called when the chip is powered on for the first time or wakes up from deepsleep.

2.2 Parameters Configuration

2.2.1 Interface

```
rf_set_default_para();
```

2.2.2 Implementation of functions

1. Set frequency, Code_Rate, bandwidth, spreading factor, transmit power and CRC

```
rf_set_freq(DEFAULT_FREQ);
rf_set_code_rate(DEFAULT_CR);
rf_set_bw(DEFAULT_BW);
rf_set_sf(DEFAULT_SF);
rf_set_crc(CRC_ON);
rf_set_tx_power(DEFAULT_PWR);
```

2.2.3 Usage

After the chip is initialized, the interface function is called in the standby3 state.

2.3 Transmitting

2.3.1 Interface

```
rf_enter_continous_tx();
rf_continous_tx_send_data();
rf_single_tx_data();
```

2.3.2 Implementation

1. Switch to STB3 state

```
rf_set_mode(RF_MODE_STB3)
```

2. Open PA

```
rf_set_ldo_pa_on
```

3. Open TX_IO

```
rf_port.set_tx
```

4. Set the transmission mode

```
rf_set_tx_mode
```

5. Enter TX state and send data

```
rf_send_packet
```

6. Get the transmission time of this packet data (optional)

```
rf_get_tx_time
```

2.3.3 Usage

After the chip is initialized and the configuration parameters are completed, the interface function is called. After the interface function is called and the chip is sent, the IRQ pin will be set high and an interrupt will be generated.

2.4 Receiving

2.4.1 Interface

```
rf_enter_continous_rx();
rf_enter_single_timeout_rx();
rf_enter_single_rx();
```

2.4.2 Implementation

1. Switch to STB3 state

```
rf_set_mode(RF_MODE_STB3)
```

2、打开 RX_IO

```
rf_port.set_rx
```

2. Open RX_IO

```
rf_set_rx_mode
```

4. Enter receiving state

```
rf_set_mode(RF_MODE_RX)
```

2.4.3 Usage

After the chip is initialized and the configuration parameters are completed, the interface function is called. After receiving, the IRQ pin will be set high. You can use polling or interrupt to determine whether the data packet is received. After receiving it, call the packet receiving function to take the data packet from the chip.

3 SDK interface description

This chapter describes in detail the user interface functions provided by the SDK.

Most parameter configurations need to be performed after the chip is initialized (in standby3 state).

3.1 rf_flag interface

Functions

```
int rf_get_recv_flag(void)
void rf_set_recv_flag(int status)
int rf_get_transmit_flag(void)
void rf_set_transmit_flag(int status)
```

Purpose

After the chip is set to send or receive, an IRQ interrupt is generated, and the event is passed to the user through the flag. The user can use the polling method to determine the chip's sending or receiving results.

Parameter

```
Current event status
RADIO_FLAG_IDLE
RADIO_FLAG_TXDONE
RADIO_FLAG_RXDONE
RADIO_FLAG_RXTIMEOUT
RADIO_FLAG_RXERR
RADIO_FLAG_PLHDRXDONE
RADIO_FLAG_MAPM
```

3.2 rf_init

Function

```
RF_Err_t rf_init(void)
```

Purpose

Initialize the chip, including wake-up, load FT configuration, initialize register configuration, AGC configuration.

Called when the chip is powered on for the first time or wakes up from deepsleep.

Return Value

FAIL Operation failed

OK Operation successful

3.3 rf_sleep_wakeup

Function

```
RF_Err_t rf_sleep_wakeup(void)
```

Purpose

Set the chip to wake up from sleep state.

When the chip is in sleep state, it is used to wake-up the chip.

When the chip is in sleep state, the register value of the chip will not be reset. After waking up from sleep, the parameter configuration before sleep is retained.

Return Value

FAIL Operation failed

OK Operation successful

3.4 rf_deepsleep

Function

```
RF_Err_t rf_deepsleep(void)
```

Purpose

Switch the chip from standby3 state to deep sleep state to reduce power consumption.

Return Value

FAIL Operation failed

OK Operation successful

3.5 rf_sleep

Function

```
RF_Err_t rf_sleep(void)
```

Purpose

Switch the chip from standby3 state to sleep state to reduce power consumption.

Return Value

FAIL Operation failed

OK Operation successful

3.6 rf_get_tx_time

Function

```
uint32_t rf_get_tx_time(uint8_t size)
```

Purpose

Get the time required to transmit this data packet. The usage of this function is demonstrated in rf_single_tx_data().

Parameter

Size: The length of the data packet to transmit

Return Value

The time required to transmit this data packet, in us.

3.7 rf_set_mode

Function

```
RF_Err_t rf_set_mode(uint8_t mode)
```

Purpose

Set the chip operation mode, including deepsleep/sleep/standby1/standby2/standby3/tx/rx.
Parameter configuration is usually performed in standby3 state.

Parameter

```
Mode Chip operation mode
RF_MODE_DEEP_SLEEP
RF_MODE_SLEEP
RF_MODE_STB1
RF_MODE_STB2
RF_MODE_STB3
RF_MODE_TX
RF_MODE_RX
```

Return Value

FAIL Operation failed

OK Operation successful

3.8 rf_get_mode

Function

```
uint8_t rf_get_mode(void)
```

Purpose

Read the current operation mode of the chip, including deepsleep/sleep/standby1/standby2/standby3/tx/rx

Return Value

Operation mode value.

3.9 rf_set_tx_mode

Function

```
RF_Err_t rf_set_tx_mode(uint8_t mode)
```

Purpose

Set the transmission mode, which can be set to single transmission or continuous transmission.

Parameter

```
Mode Transmit data mode
RF_TX_SINGLE
RF_TX_CONTINOUS
```

Return Value

FAIL Operation failed

OK Operation successful

3.10 rf_set_rx_mode

Function

```
RF_Err_t rf_set_rx_mode(uint8_t mode)
```

Purpose

Set the receiving mode, which can be set to single receiving, timeout receiving, continuous receiving.

Parameter

Mode Receive mode
RF_RX_SINGLE
RF_RX_SINGLE_TIMEOUT
RF_RX_CONTINOUS

Return Value

FAIL Operation failed

OK Operation successful

3.11 rf_set_rx_single_timeout

Function

```
RF_Err_t rf_set_rx_single_timeout(uint16_t timeout)
```

Purpose

Set the timeout period for timeout reception. It is only valid in timeout reception mode.

Parameter

Timeout 1~65535 Timeout period, in ms

Return Value

FAIL Operation failed

OK Operation successful

3.12 rf_get_snr

Function

```
float rf_get_snr(void)
```

Purpose

Read the value of the signal-to-noise ratio SNR. The higher the SNR, the better the signal quality. This interface needs to be called when a data packet is received and before the RxDone interrupt is cleared. If the interrupt is cleared, this value will become invalid. The same is true for RSSI. The rf_irq_process() function provides a demonstration of the use of this function.

Example:

```
// RxDone IRQ received
  snr = rf_get_snr();
  rssi = rf_get_rssi();
  rf_receive(buf);
```

Return Value

SNR value

3.13 rf_get_rssi

Function

```
int8_t rf_get_rssi(void)
```

Purpose

Read the received signal strength value. This interface needs to be called when a data packet is received and before the RxDone interrupt is cleared. If the interrupt is cleared, this value will become invalid. The same is true for SNR. The rf_irq_process() function provides a usage demonstration of this function. For more usage, please refer to the RSSI application reference document.

Example:

```
// RxDone IRQ received
  snr = rf_get_snr();
  rssi = rf_get_rssi();
  rf_receive(buf);
```

Return Value

RSSI value

3.14 rf_get_channel_rssi

Function

```
int8_t rf_get_channel_rssi(void)
```

Purpose

PAN3029/3060 supports channel energy detection function, and the channel rssi value can be used to indicate the energy level of the channel. After the chip enters the RX state, this function can be called to obtain the channel detection energy and read the channel energy strength value.

Example:

```
//After the chip enters the RX state
channel_rssi = rf_get_channel_rssi();
```

Return Value

RSSI value

3.15 rf_get_irq

Function

```
uint8_t rf_get_irq(void)
```

Purpose

Read interrupt register value. Used in rf_irq_process.

Return Value

Interrupt flags.

3.16 rf_clr_irq

Function

```
RF_Err_t rf_clr_irq(uint8_t flags)
```

Purpose

Clear the corresponding interrupt.

Parameter

```
Interrupt flags
REG_IRQ_MAPM_DONE
```

REG_IRQ_RX_PLHD_DONE

REG_IRQ_RX_DONE

REG_IRQ_CRC_ERR

REG_IRQ_RX_TIMEOUT

REG_IRQ_TX_DONE

Return Value

FAIL Operation failed

OK Operation successful

3.17 rf_set_refresh

Function

RF_Err_t rf_refresh(void)

Purpose

Register parameters are refreshed. Register values are not modified.

Return Value

FAIL Operation failed

OK Operation successful

3.18 rf_set_preamble

Function

RF_Err_t rf_set_preamble(uint16_t reg)

Purpose

Setting the chip to transmit the preamble is only valid for the transmitter and needs to be configured before transmitting data. When transmitting data, the transmission content consists of "preamble + data". The longer the preamble is, the longer the transmission time of the entire transmission content is.

Parameter

8~65535 Transmit preamble length

Return Value

FAIL Operation failed

OK Operation successful

3.19 rf_cad_on

Function

RF_Err_t rf_cad_on(uint8_t threshold, uint8_t chirps)

Purpose

Set the chip to turn on the CAD function. This function modifies the receiving threshold before the chip enters the receiving mode.

Parameters

Receiving threshold

CAD_DETECT_THRESHOLD_0A

CAD_DETECT_THRESHOLD_10

CAD_DETECT_THRESHOLD_15

CAD_DETECT_THRESHOLD_20

Number of chirps symbol detections

CAD_DETECT_NUMBER_1

CAD_DETECT_NUMBER_2

CAD_DETECT_NUMBER_3

Return Value

FAIL Operation failed

OK Operation successful

3.20 rf_cad_off

Function

RF_Err_t rf_cad_off(void)

Purpose

Set the chip to turn off the CAD function and restore the receiving threshold.

Return Value

FAIL Operation failed

OK Operation successful

3.21 rf_set_syncword

Function

```
RF_Err_t rf_set_syncword(uint8_t sync)
```

Purpose

Set the chip's synchronization word.

Parameter

Sync	Synchronization word

Return Value

FAIL Operation failed

OK Operation successful

3.22 rf_get_syncword

Function

```
uint8_t rf_get_syncword(void)
```

Purpose

Read the chip's sync word.

Return Value

Sync word value

3.23 rf early interrupt interface

Functions

```
RF_Err_t rf_set_plhd(uint8_t addr, uint8_t len)
void rf_set_plhd_rx_off(void)
uint8_t rf_get_plhd_len(void)
RF_Err_t rf_plhd_receive(uint8_t *buf, uint8_t len)
```

Purpose

The early interrupt function is to check the decoded data during the chip reading a frame of data, determine whether it is what you want, and then decide whether to continue reading or abandon this frame of data. For more function descriptions, please refer to the "PAN3029_3060 Series Early Interrupt Application Reference Document".

3.24 rf_set_mapm_rx_on

Function

```
void rf_set_mapm_on(void)
```

Purpose

Enable mapm mode and mapm interrupt

Return Value

None

3.25 rf_set_mapm_rx_off

Function

```
void rf_set_mapm_off(void)
```

Purpose

Disable mapm mode and mapm interrupt

Return Value

None

3.26 rf_set_mapm_para

Function

void rf_set_mapm_para(stc_mapm_cfg_t *p_mapm_cfg)

Purpose

Set mapm mode related parameters

Parametres

p_mapm_cfg->fn is used to configure the number of fields in mapm preamble

p_mapm_cfg->fnm is used to configure the number of repetitions of each field in mapm preamble

p_mapm_cfg->gfs is used to configure the last group in the field, its ADDR position function selection

p_mapm_cfg->gn is used to configure the number of groups in a field

p_mapm_cfg->pgl is used to configure the number of preambles in the first group

p_mapm_cfg->pgn is used to configure the number of preambles in other groups

p_mapm_cfg->gn is the number of chirps before the syncword after all fields are sent

Return Value

None

3.27 rf_receive

Function

```
uint8_t rf_recv_packet(uint8_t *buff)
```

Purpose

Receive a packet of data

Parameters

Buff - buffer used to receive data packets

Return Value

Length of received data packet

3.28 rf_enter_continous_rx

Function

```
RF_Err_t rf_enter_continous_rx(void)
```

Purpose

Set the chip to enter continuous receiving mode.

In this mode, the chip will always be in receiving state. When data is received, the chip will generate rx IRQ related interrupts and continue to receive.

Return Value

FAIL Operation failed

OK Operation successful

3.29 rf_enter_single_timeout_rx

Function

```
RF_Err_t rf_enter_single_timeout_rx(uint32_t timeout)
```

Function

Set the chip to enter the timeout receiving mode.

In this mode, the chip will receive once. After receiving the data, the chip will generate an rxdone IRQ interrupt and wait for the user's subsequent operation. When no data is received within the timeout period, the chip will generate an rxtimeout IRQ interrupt and wait for the user's subsequent operation. If you give up receiving, you need to actively exit the receiving state, otherwise the chip will start a timeout reception again. After use, the user software must first switch to STB3 to exit the receiving state before performing subsequent operations.

Parameter

Timeout

Return Value

FAIL Operation failed

OK Operation successful

3.30 rf_enter_single_rx

句法

RF_Err_t rf_enter_single_rx(void)

目的

设置芯片进入单次接收模式接收。

该模式下，芯片会进行一次接收，当接收到数据后，芯片会产生 rx IRQ 相关中断，并退出接收状态，用户软件需先切换至 STB3 退出接收状态，再进行后续操作。

返回值

FAIL 操作失败

OK 操作成功

3.31 rf_single_tx_data

句法

RF_Err_t rf_single_tx_data(uint8_t *buf, uint8_t size, uint32_t *tx_time)

目的

设置芯片进入单次发射模式并发射数据。

该模式下，芯片会进行一次发射，当发射完成后，芯片会产生 txdone IRQ 中断，并退出发射状态（工作电流降低），用户软件需先切换至 STB3 退出发射状态，再进行后续操作。

参数

Buf 发送 buff

Size　发送包长

Tx_time 获取本次数据包的传输时间，tx_time 参数非必须，用户可根据需要删减

返回值

FAIL　操作失败

OK　　操作成功

3.32 rf_enter_continous_tx/rf_continous_tx_send_data

句法

```
RF_Err_t rf_enter_continous_tx(void)
RF_Err_t rf_continous_tx_send_data(uint8_t *buf, uint8_t size)
```

目的

设置芯片进入连续发射模式并发射数据。

该模式下，芯片一次发射完成后，芯片会产生 txdone IRQ 中断，并保持发射状态（工作电流仍为发射电流，但无数据发射），并等待用户后续操作。

随后，用户可以选择停止发射，将芯片状态切换至 standby3；或者选择继续发射，直接调用 rf_continous_tx_send_data()即可，继续发射不需要重复 rf_enter_continous_tx()操作。

参数

Buf　发送 buff

Size　发送包长

返回值

FAIL　操作失败

OK　　操作成功

3.33 rf_set_agc

句法

```
RF_Err_t rf_set_agc(bool NewState)
```

目的

配置芯片 AGC。默认初始化芯片时配置并打开 AGC。

参数

State　AGC_OFF/ AGC_ON

返回值

FAIL　操作失败

OK　操作成功

3.34 rf_set_dcdc_mode

句法

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
```

目的

设置芯片 DCDC 模式，默认关闭。DCDC 模式可以获得更低的接收电流。

开启本功能前，请联系并确认模组是否支持该功能开启，否则易对模组造成损害。

参数

Dcdc_val　模式类型

DCDC_ON

DCDC_OFF

返回值

FAIL　操作失败

OK　操作成功

3.35 rf_set_ldr

句法

```
RF_Err_t rf_set_ldr(uint32_t mode)
```

目的

设置芯片 LDR 低速率模式，默认关闭。低速率模式可以进一步降低传输速率，获得更好的传输效果。

参数

Mode　模式类型

LDR_ON

LDR_OFF

返回值

FAIL　操作失败

OK　操作成功

3.36 rf intelligent search interface

句法

```
RF_Err_t rf_set_auto_sf_tx_preamble(int sf, int sf_range[], int size, int chirp_counts[])
RF_Err_t rf_set_auto_sf_rx_on(int sf_range[], int size)
RF_Err_t rf_set_auto_sf_rx_off(void)
int calculate_chirp_count(int sf_range[], int size, int chirp_counts[])
```

目的

智能搜索功能，可实现在接收时智能化识别信道中的 SF 参数，达到接收不同 SF 信号数据的目的。更多功能说明请参考《PAN3029_3060 系列_智能搜索应用参考文档》。

3.37 get_chirp_time

句法

```
uint32_t rf_get_chirp_time(uint8_t bw, uint8_t sf)
```

目的

计算单个 chirp 持续时间。

参数

SF：SF_5/SF_6/SF_7/SF_8/SF_9/SF_10/SF_11/SF_12

BW：BW_62_5K/BW_125K/BW_250K/BW_500K

返回值

单个 chirp 持续时间，单位：us。

3.38 check_cad_rx_inactive

句法

```
bool check_cad_rx_inactive(uint32_t one_chirp_time)
```

目的

使用 CAD 检测当前信道状态，用于 CAD 接收前的信道检测。

参数

one_chirp_time：单个 chirp 持续时间

返回值

LEVEL_ACTIVE　　信道活跃，应当继续接收

LEVEL_INACTIVE　信道无信号，应当放弃接收

4 SDK Porting

本章节详细描述 SDK 移植过程及注意事项。keil 配置 floating point hardware 为“Not Used”，配置“One ELF Section per Function”不勾选，优化等级设置为“default”。编译代码后，统计 SDK 所需的 Flash 大小为 9125 (8.91kB)，所需 SRAM 大小为 1485 (1.45kB)。

SDK《pan_port.c》中提供了用户需要移植的接口函数。要正确运行 SDK，需要实现以下接口：

- spi_cs_set_high()，SPI 片选拉高。
- spi_cs_set_low()，SPI 片选拉低。
- spi_readwritebyte()，SPI 数据传输。
- rf_delay_ms()，rf_delay_us()，delay 函数。
- RX_IO/TX_IO 实现，芯片收发 IO 切换接口，包括 rf_antenna_init，rf_antenna_tx，rf_antenna_rx，rf_antenna_close。

  RX_IO/TX_IO 实现，SDK 中通过 SPI 读写寄存器实现该部分，通过 SPI 控制芯片内部 GPIO4 和 GPIO3 实现 RX_IO/TX_IO。

  用户也可以使用外部 MCU IO 去控制 RX_IO/TX_IO，具体实现取决于底板设计情况。
- TCXO_IO 实现，芯片有源晶振 IO 控制接口，包括 rf_tcxo_init，rf_tcxo_close。

  TCXO_IO 实现，SDK 中通过 SPI 读写寄存器实现该部分，通过 SPI 控制芯片内部 GPIO5 实现 TCXO_IO。

  用户也可以使用外部 MCU IO 去控制 TCXO_IO，具体实现取决于 PAN3029/3060 系列模组版本和 MCU 底板设计情况。

5 PAN3029/3060 Series Status Diagram

本章节描述的是 PAN3029/3060 系列芯片的状态切换图，方便用户深入了解芯片工作原理。其中，DeepSleep 到 STB3(standby3)的唤醒过程及配置初始化对应 SDK 中的 rf_init 函数；STB3 到 DeepSleep 对应 SDK 中的 rf_deepsleep 函数。Sleep 到 STB3 的切换过程对应 SDK 中的 rf_sleep_wakeup 函数；STB3 到 Sleep 对应 SDK 中的 rf_sleep 函数。

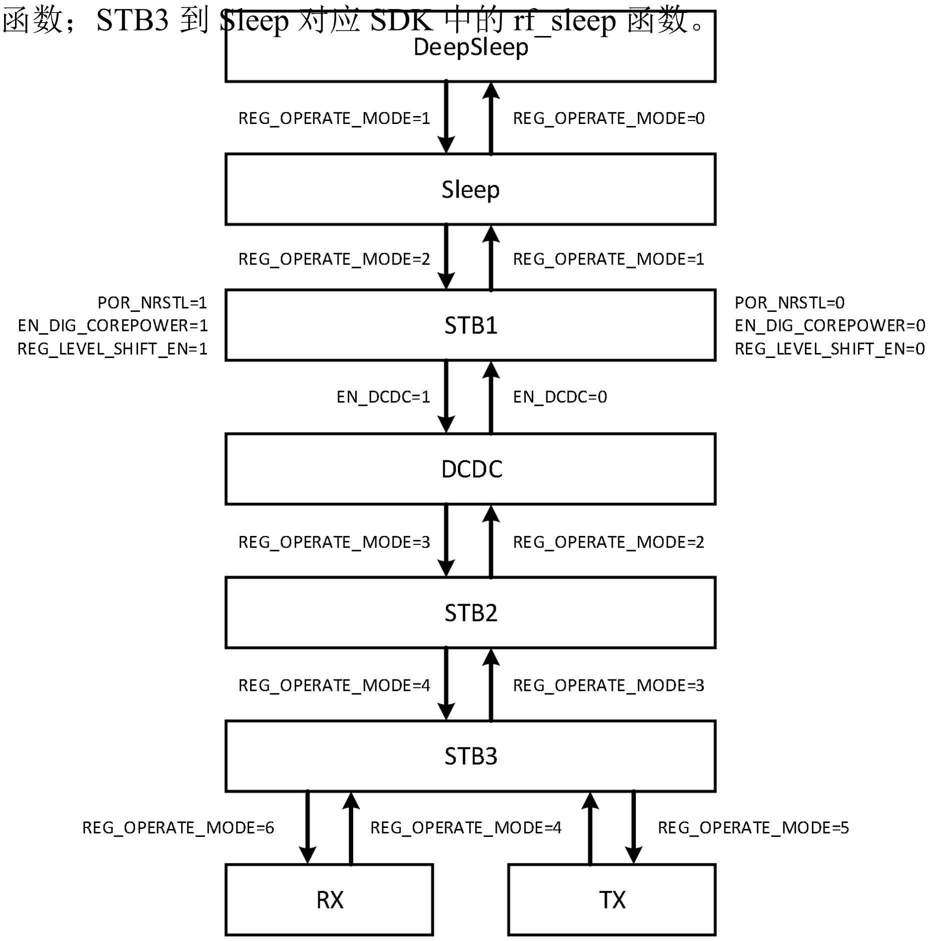

图 6-1 PAN3029 上电流程

PAN3029/3060 系列芯片上电后有七种状态，包括深度睡眠模式(DeepSleep)、睡眠模式(Sleep)、LDO 工作模式(STB1)、OSC 工作模式(STB2)、OSC 输出模式(STB3)、TX 模式以及 RX 模式。所有状态可通过配置寄存器进行切换。

注：如果芯片外围电路不支持 DCDC，可不配置 EN_DCDC，直接从 STB1 进入 STB2。SDK 默认不使用 DCDC 功能，如果需要使用 DCDC 功能，请参考对应的应用文档。

6 Application Notes

6.1 Low Power Sleep

关于低功耗休眠，SDK 中提供了四个接口函数。

一种是睡眠 Sleep 模式。包含从 STB3 到 Sleep 的 rf_sleep 休眠函数，从 Sleep 到 STB3 的 rf_sleep_wakeup 唤醒函数。

一种是深度休眠 DeepSleep 模式。包含从 STB3 到 DeepSleep 的 rf_deepsleep 休眠函数，从 DeepSleep 到 STB3 的 rf_init 唤醒函数。注意，PAN3029/3060 系列芯片芯片进入深度休眠并再次唤醒后，原有的配置参数需要重新配置，收发模式也需要重新选择配置。

6.2 CAD Function

CAD 功能，修改了芯片的接收阈值，可能影响芯片的接收灵敏度。

使用完成后，需要关闭 CAD 功能。

更多功能说明请参考《PAN3029_3060 系列_CAD 应用参考文档》。

6.3 Sensitivity and RSSI

读取信号强度值需要在接收到数据包的时候调用，且在清除 rxdone 中断之前。如果清除中断，这个值就会失效。RSSI 的测量范围是-12 到-110，不同参数（SF、BW）模式下，测量范围略有不同。

当需要读取芯片 RSSI 时，建议打开 AGC，可以提高芯片 RSSI 准确性。更多功能说明请参考《PAN3029_3060 系列_RSSI 应用参考文档》。

6.4 DCDC Function

DCDC 功能开启后，可以使芯片的接收电流降低至 4.77mA，默认该功能关闭。SDK 中提供了一个接口函数开启或关闭 DCDC 功能。

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
参数 dcdc_val
 DCDC_ON      开启
 DCDC_OFF     关闭
```

DCDC 功能可在 STB1~STB3 及 RX 模式开启。

开启 DCDC 功能后，当芯片需要发送时，需要提前手动关闭 DCDC 功能。

开启 DCDC 功能前，请联系并确认模组是否支持该功能开启，否则易对模组造成损害。

6.5 EFUSE Function

PAN3029/3060 系列芯片支持 82Bytes 的 eFuse 空间区域供客户使用，可用于存储掉电不丢失的数据。关于 eFuse 功能的使用，SDK 中提供了四个接口函数。

```
uint32_t rf_efuse_on(void)
目的
设置芯片打开 eFuse 功能
参数
无
返回值
FAIL   操作失败
OK     操作成功


uint32_t rf_efuse_off(void)
目的
设置芯片关闭 eFuse 功能
```

参数

无

返回值

FAIL　操作失败

OK　　操作成功

uint8_t rf_efuse_read_byte(uint8_t reg_addr, uint8_t efuse_addr)

目的

读取指定 efuse 地址存储数据

参数

reg_addr　efuse 进入地址，客户区默认 0x3c

efuse_addr　efuse 内部地址，客户区 0x2d~0x7f

返回值

value　读取指定 efuse 地址返回的数据

void rf_efuse_write_byte(uint8_t reg_addr, uint8_t efuse_addr, uint8_t value)

目的

往指定 efuse 地址写入想存储的数据

参数

reg_addr　efuse 进入地址，客户区默认 0x3c

efuse_addr　efuse 内部地址，客户区 0x2d~0x7f

value　欲写入 efuse 的数据

返回值

无

eFuse 读写需要特别注意：

1.eFuse 只能烧写一次，一旦进行烧写，无法修改。

2.不能对 eFuse 同一地址重复写，否则容易导致 eFuse 器件损坏。

3.写 eFuse 的电压范围是 2.25V 到 2.75V，典型值为 2.5V，因此在配置 eFuse 时，建议外部

VDD1 使用 2.5V 电压。读电压可正常使用 3V 电压。

4.eFuse 仅支持在 STB3 读写操作，请勿在其他模式下读写。

5.eFuse 区域不支持 SPI 和 I2C 的连续读写操作。

6.eFuse 读写的最高时钟速率为 8MHz。

6.6 RF Parameters Description

- **信道带宽（BW）**

信道带宽是限定允许通过该信道的信号下限频率和上限频率，可以理解为一个频率通带，其取值范围为 62.5KHz、125KHz、250KHz、500KHz。比如一个信道允许的通带为 1.5kHz 至 15kHz，则其带宽为 13.5kHz。增加 BW，可以提高有效数据速率以缩短传输时间，但是以牺牲部分接受灵敏度为代价，从而影响通信距离。

- **扩频因子（SF）**

SF 取值范围 SF5-12，值越小传输速率越高，传输距离越短，相反 SF 值越大，传输速率越慢，传输距离越远。在同等数量的数据传输时，SF 越大传输时间越长。

- **编码率（CR）**

CR 是数据流中有用部分（非冗余）的比例，取值范围 1-4 对应 1=4/5，2=4/6，3=4/7，4=4/8。也就是说，如果编码率是 k/n，则对每 k 位有用信息，编码器总共产生 n 位的数据，其中 n-k 是多余的。CR 越大单次传输的多余数据越多，有效数据速率降低，传输时间变长。

- **校验（CRC）**

CRC 是 TX 端设置，RX 端不设置。RX 端接收信号中的信息，在帧结构中读取 CRC，来决定要不要进行 CRC 检查。

- **低速率模式**

取值范围 1 或者 0，为 1 时为开启低速率模式，默认为关闭，开启后通信速率会降低，接收灵敏度会提升 1-2dBm，通信距离会稍增加。

- **前导码符号数量**

取值范围 8—65535，默认值为 8。在实现空中唤醒应用场景时可以将前导码符号数量增大，同时发射时间也会增大，从而实现接收端在低功耗模式下被长的前导码唤醒的功能。

备注：上述 RF 参数需要根据实际应用场景进行选择配置，且收发端的 RF 配置需要一致时才能正常通信。RF 数据发送时间及参数的选择可参考 SDK 中的计算工具。

6.7 4-wire SPI and FIFO

外部设备可通过四线 SPI 方式对芯片中的寄存器和 FIFO 进行配置访问。

PAN3029/3060 系列芯片实现了 SPI 总线的从机 Slave，用于读写寄存器和 FIFO。SPI 总线为四线制，分别为：

- SCK（时钟）
- CSN（片选信号，低电平有效）
- MOSI（数据输入）
- MISO（数据输出）

其中 SCK、CSN、MOSI 由主机 Master 控制，MISO 由 Slave 控制。

在通信过程中，以 CSN 电平拉低起始，直至 CSN 电平拉高时结束本次传输过程。主机 Master 通过 MOSI 发送数据，MISO 接收数据。SCK 下降沿时产生数据，上升沿时进行数据采样。

Master 传输的信息由 Address Byte 和 Data Byte 两部分组成。其中 Address Byte 前 7bit 为地址位 addr；最后 1bit 为读写位 wr，写操作时该 bit 置 1，读操作时该 bit 置 0。

SPI 有三种传输模式：

- Single：单字节传输模式。信息仅为 2 byte，Master 通过 MOSI 发送 Address Byte。若为写操作，Master 继续通过 MOSI 发送 Data Byte；若为读操作，则 Master 读取 MISO 上 Slave 回复的 Data Byte。
- Burst：突发连续传输模式。信息大于 2byte，Address Byte 后跟若干个 Data Byte，Data Byte 之间无需增加 Address Byte，从机 Slave 内部会自动在每个 Data Byte 之间递增地址。CSN 信号在最后一个 Data Byte 后拉高，其余传输信息过程均维持低电平。
- FIFO：FIFO 读写模式。该模式下单字节或连续传输均可实现，传输规则同 Signle 模式和 Burst 模式，不同点在于 Address Byte 中的地址位 addr 只能配置为 7’h1，且 Slave 在 Data Byte 之间不做地址递增操作。

  SPI 写时序如下：

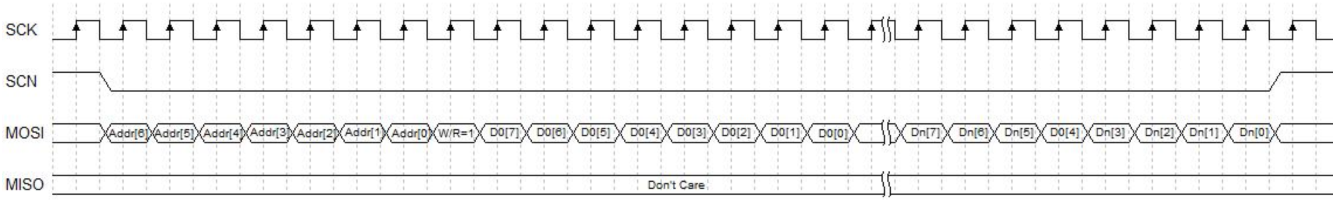

图 7-1 SPI 写时序

SPI 读时序如下：

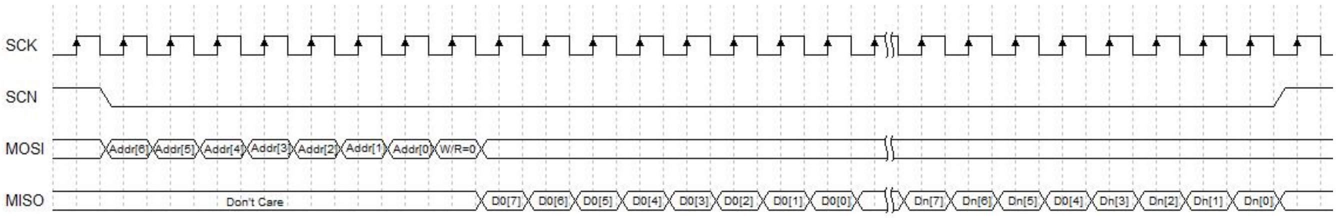

图 7-2 SPI 读时序

PAN3029/3060 系列芯片具有 255bytes 的 FIFO 用以存储 TX 模块发送数据和 RX 模块解码数据。

FIFO 由单口 RAM 组成，只能实现单包数据信息的存储和读取，在 FIFO 已存有一包数据的情况下，应先读取完此包数据后再写入，否则 FIFO 中前一包数据将被覆盖。

7 Appendix

7.1 Transmitting power table

频率 / 档位	410MHz		430MHz		450MHz		460MHz	
	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)
1	-18.733	7.267	-18.165	7.418	-18.767	7.546	-19.679	7.607
2	-8.276	9.42	-7.936	9.563	-8.596	9.671	-9.548	9.758
3	1.599	17.935	1.748	18.349	0.947	18.725	-0.111	18.932
4	3.925	20.301	4.513	20.661	4.114	20.967	3.348	21.439
5	4.921	21.76	5.329	22.183	4.79	22.662	3.865	23.076
6	5.413	22.933	5.501	23.385	4.776	23.811	3.841	24.136
7	7.104	26.605	7.611	26.904	7.345	27.468	6.805	28.54
8	7.655	28.062	8.152	28.282	7.951	28.995	7.491	30.298
9	8.638	30.074	9.185	30.663	8.755	31.328	8.117	32.387
10	8.966	32.23	9.29	31.92	9.106	32.679	8.897	34.62
11	10.274	35.676	10.746	36.083	10.571	37.273	10.298	39.578
12	10.816	39.495	10.988	38.391	10.753	38.94	10.56	41.211
13	11.567	41.972	11.815	41.256	11.612	42.165	11.468	44.97
14	12.83	46.616	13.205	46.748	13.077	48.484	12.919	52.149
15	13.782	50.643	14.252	51.595	14.182	54.016	14.001	58.272
16	14.871	55.72	15.404	57.538	15.36	60.671	15.015	64.89
17	15.404	58.355	15.957	60.647	15.797	63.599	15.317	67.146
18	16.052	63.254	16.602	65.475	16.522	69.186	16.267	74.527
19	16.493	66.892	16.956	68.616	16.899	72.499	16.738	78.608
20	17.414	73.954	17.868	75.84	17.8	80.261	17.632	86.882
21	18.229	87.976	18.297	85.286	17.994	86.519	17.694	90.391
22	19.436	102.128	19.414	98.008	19.07	98.768	18.736	102.895
23	19.754	105.378	19.771	101.687	19.433	102.835	19.105	107.278

注：功率挡位为 rf_set_tx_power 函数的参数。
其它频段的发射功率和发射电流会略有不同。